

SARKARI SCHEME FINDER

Government Scheme Eligibility & Recommendation Portal

Complete System Architecture, Directory Structure & Feature-Technology Mapping

Author / Developer: Yadam Gopikrishna

Technology Stack: MERN Stack (MongoDB, Express.js, React 18, Node.js) + Vite + Tailwind CSS

Document Scope: Standalone Technical Specification & Codebase Reference Guide

1. Executive Overview

"Sarkari Scheme Finder" is a full-stack, citizen-centric government welfare recommendation portal designed to bridge the awareness gap for 1.4 billion Indian citizens. The platform eliminates bureaucratic complexity by allowing citizens to enter basic demographic, geographic, and economic details to instantly discover relevant Central, State, and Union Territory welfare initiatives.

Core Architectural Pillars:

- Multi-Tier Eligibility Engine:** Evaluates citizen age, gender, state residence, annual income, occupation, student status, disability benchmarks, and agricultural landholding against structured MongoDB rules.
- Multilingual Support (8 Indian Languages):** Instant dynamic UI switching across English, Telugu (ÀLFÃ,AÁ|A), Hindi ("™?'%M'i@), Kannada (É\Û'É 'Â Ò Æ yalam (Òí2Òy>Ó=), Marathi ('é0"é " 'Â æB &Væv Æ' %o→é,)>¼).
- Grounded AI Assistant:** Dual-mode conversational AI with intent classification, verified scheme data grounding, official helplines, application procedures, and optional Google Gemini LLM connectivity.
- Privacy-First Zero Storage Policy:** Strict non-negotiable architectural design: Zero collection, storage, or transmission of Aadhaar numbers or biometrics.
- Direct Government Links:** Eliminates intermediary corruption by routing citizens directly to authenticated .gov.in and .nic.in portals.

2. System Architecture High-Level Overview

The portal follows a Decoupled Single-Page Application (SPA) architecture communicating over secure JSON REST APIs:

- Frontend Tier:** React 18 single-page application built on Vite 6, styled with Tailwind CSS, utilizing React Context for global Auth and Language state management.
- Backend Tier:** Node.js & Express.js REST API with modular controllers, services, middleware, security headers (Helmet), and strict request rate-limiting.
- Database Tier:** MongoDB Community Server with Mongoose 8 ODM schemas, compound indexes for fast state/category filtering, and audit logging.

3. Complete Project Directory Tree

Below is the complete filesystem placement of every folder and configuration file across the Sarkari Scheme Finder repository:

```
fsd-2 project/ (Root Directory)
% % % package.json           # Root orchestrator & development scripts (concurrently)
% % % package-lock.json      # Root dependency lockfile
% % % .gitignore             # Git exclusions (node_modules, .env, build artifacts)
% % % README.md              # Comprehensive project documentation & startup guide
% % % PRESENTATION.md        # 13-Slide presentation deck outline & speaker notes
% % % PROJECT_STRUCTURE_AND_TECH_STACK.pdf # This technical reference specification
%
% % % server/                # Backend Server Directory (Port 5000)
% % % % package.json         # Backend dependencies (Express, Mongoose, JWT, etc.)
% % % % server.js            # Main Express application entrypoint & port handlers
% % % % .env                 # Local environment variables (Port, MongoDB URI, Secrets)
% % % % .env.example         # Safe repository template for environment setup
% % % % config/
% % % % % db.js              # Mongoose connection manager with error handling
% % % % models/
% % % % % % User.js          # Citizen & Admin schema with bcrypt encryption
% % % % % % Scheme.js        # Comprehensive welfare scheme rules & criteria schema
% % % % % % Category.js      # Scheme category taxonomy schema
% % % % % % StateDistrict.js # 36 States/UTs with official district mappings schema
% % % % % % SavedScheme.js    # Citizen bookmarking & saved scheme records schema
% % % % % % EligibilityCheck.js # Citizen eligibility query audit logging schema
% % % % % % Notification.js   # Platform broadcast announcements & deadline alerts schema
% % % % controllers/
% % % % % % authController.js # Register, Login, Profile update, Password reset
% % % % % % schemeController.js # Scheme browsing, filtering, search, and detail retrieval
% % % % % % eligibilityController.js # Multi-parameter calculation & query audit logging
% % % % % % savedSchemeController.js # Bookmark management for logged-in citizens
% % % % % % adminController.js # Admin scheme CRUD, verification updates & analytics
% % % % % % metaController.js # States, UTs, districts, and categories metadata endpoints
% % % % % % notificationController.js # System announcements & mark-as-read endpoints
% % % % % % aiController.js   # AI Chatbot endpoint handler (/api/chat)
% % % % services/
% % % % % % eligibilityEngine.js # 7-step rule matching & percentage calculation engine
% % % % % % aiAssistantService.js # Grounded scheme AI assistant & intent resolution
% % % % middleware/
% % % % % % authMiddleware.js # JWT token verification & citizen session guard
% % % % % % adminMiddleware.js # Role-based access guard for admin routes
% % % % % % errorMiddleware.js # Centralized 404 & 500 exception handler
% % % % seed/
% % % % % % seedData.js       # 36 States/UTs, 11 categories, 23 realistic schemes
% % % % % % runSeed.js        # Automated database seeding runner script
%
% % % client/                # Frontend Application Directory (Port 5173)
% % % % package.json         # Frontend dependencies (React 18, Tailwind, Lucide, Axios)
% % % % vite.config.js       # Vite 6 config with 0.0.0.0 host & /api proxy routing
% % % % tailwind.config.js   # Custom government theme palette (Sarkari Navy, Saffron)
% % % % postcss.config.js    # PostCSS configuration for Tailwind CSS
% % % % index.html           # Single-page HTML entrypoint with Ashoka Chakra icon
% % % % src/
% % % % % % main.jsx          # React DOM root entrypoint & Context providers
% % % % % % App.jsx           # React Router v6 routing & layout definitions
% % % % % % index.css         # Global styles, Tailwind directives, custom scrollbars
% % % % % context/
% % % % % % % AuthContext.jsx # User session, JWT storage, Login/Logout state
% % % % % % % LanguageContext.jsx # 8-language localization provider & persistence
% % % % % services/
% % % % % % % api.js          # Axios client with JWT interceptor & ngrok header
% % % % % translations/
% % % % % % % translations.js # Complete 8-language UI translation dictionary
% % % % % components/
% % % % % % % Navbar.jsx      # Sticky navigation, language switcher, auth status
% % % % % % % Footer.jsx      # Government disclaimers, links, national helplines
% % % % % % % SchemeCard.jsx   # Scheme card with category badge, match pill, actions
% % % % % % % AiAssistantModal.jsx # Floating AI chatbot with markdown & quick prompts
% % % % % % % NotificationDropdown.jsx # Real-time platform announcements dropdown
% % % % % % % LanguageSelector.jsx # Accessible 8-language dropdown selector
% % % % % % % ProtectedRoute.jsx # Route guard for logged-in citizens
% % % % % % % AdminRoute.jsx  # Route guard restricted to administrators
% % % % pages/
% % % % % % % HomePage.jsx     # Hero search, category tiles, statistics, recent schemes
% % % % % % % BrowseSchemesPage.jsx # Filterable scheme directory with search & pagination
% % % % % % % SchemeDetailsPage.jsx # In-depth view with rules, docs & verified apply button
% % % % % % % EligibilityFormPage.jsx # 7-step accessible wizard with conditional logic
% % % % % % % ResultsPage.jsx  # Matched schemes with percentage scores & condition checks
% % % % % % % CompareSchemesPage.jsx # 4-way side-by-side scheme comparison matrix
```

```
% % % SavedSchemesPage.jsx # Citizen saved schemes dashboard
% % % DashboardPage.jsx   # Citizen overview, recent queries & bookmark summary
% % % UserProfilePage.jsx # Edit demographic profile & state preference
% % % AdminDashboardPage.jsx # Scheme CRUD, verification timestamps & analytics
% % % LoginPage.jsx       # Citizen & Admin authentication page
% % % RegisterPage.jsx    # Citizen account registration page
% % % ForgotPasswordPage.jsx # Self-service password recovery page
% % % NotFoundPage.jsx    # Custom 404 error page with navigation shortcuts
```

4. Feature-to-Technology Stack Mapping

The matrix below details which specific technology, library, or framework was chosen for each core capability of the Sarkari Scheme Finder portal, along with its architectural implementation rationale:

Feature / Capability	Technology Stack	Architectural Role & Implementation Detail
7-Step Eligibility Wizard	React 18 + Tailwind	Step-by-step form state management, real-time client validation, and dynamic conditional rendering (e.g. agricultural landholding step shown
Rule Engine & Weighted Matching	Node.js (Custom Service)	Multi-criteria scoring algorithm in eligibilityEngine.js evaluating age, income, state, gender, student, PwD, and land rules. Outputs match percentage and
Grounded AI Scheme Assistant	Express + MongoDB + Gemini	Regex intent classification + MongoDB text query grounding. Integrates optional Google Gemini API while falling back to local knowledge engine for
8-Language Multilingual UI	React Context + JSON Dict	Zero-dependency localization engine in translations.js. Supports English, Telugu, Hindi, Tamil, Kannada, Malayalam, Marathi, and Bengali with
Scheme Comparison Tool	React State + CSS Grid	Side-by-side comparison matrix of up to 4 schemes across criteria, financial benefits, income ceilings, age limits, and required documents.
Authentication & RBAC	JWT + bcryptjs	Stateless token authentication. Passwords hashed with 10 salt rounds. ProtectedRoute.jsx and AdminRoute.jsx enforce role-based access for
Backend REST API	Express.js 4.21	Modular MVC routing architecture with dedicated controllers, robust parameter validation, and clean separation between business logic and
Database & Schemas	MongoDB + Mongoose 8.9	Schema validation with compound indexes on state, category, and status. Audit logging of citizen queries and scheme view counter increments.
Frontend UI & Design System	Tailwind CSS + Lucide Icons	Government-themed palette (Navy #0F172A, Saffron #EA580C, Emerald #059669). Fully responsive mobile-first layouts, accessible contrast, and
Frontend Bundler & Dev Server	Vite 6.4 (ESBuild)	Sub-second hot module replacement (HMR), tree-shaking, production chunking, and built-in reverse proxy routing (/api -> http://localhost:5000).
HTTP Client & Interceptors	Axios 1.7	Configured in api.js with automated Authorization Bearer header injection and ngrok-skip-browser-warning headers to prevent interstitial blocking.
Security Headers & Rate Limiting	Helmet 8.0 + Rate-Limit	Sets HTTP security headers (CSP, HSTS, X-Frame-Options) and rate-limits API endpoints to prevent abuse and denial-of-service attempts.
Automated Database Seeding	Node.js (runSeed.js)	Populates 36 States/UTs, 11 categories, 23 realistic Central & State schemes (PM-KISAN, PMAY, PM-JAY, AP Thalliki Vandanam, etc.), and
Process Orchestration	Concurrently 9.1	Enables unified single-command execution (npm run dev) running both backend server (port 5000) and frontend client (port 5173) simultaneously.
Public Tunneling & Demo	Ngrok HTTP Tunnel	Allows secure remote demonstration of the local development portal on mobile devices and remote networks with host header whitelisting in Vite.
Citizen Privacy & Compliance	Architectural Policy	Strict zero-storage policy for Aadhaar numbers and biometric data. Informational disclaimer compliance; official external redirection to .gov.in websites.

5. Deep Dive: Folder Responsibilities & Structure

Server Directory Structure (server/):

- **server/config/db.js:** Encapsulates Mongoose database connection initialization, event listeners for disconnect/reconnect, and graceful connection shutdown.
- **server/models/:** Houses 7 Mongoose schemas with strict data typing, default values, and index definitions (User, Scheme, Category, StateDistrict, SavedScheme, EligibilityCheck, Notification).
- **server/services/:** Contains core business logic isolated from HTTP concerns: eligibilityEngine.js (scoring algorithm) and aiAssistantService.js (intent analysis and database lookup).
- **server/controllers/:** Executes request parsing, invokes domain services, queries Mongoose models, and formats standardized JSON API responses ({ success, data, message }).
- **server/routes/:** Maps REST endpoints to controller methods with applied middleware (e.g. authRoutes, schemeRoutes, eligibilityRoutes, aiRoutes).
- **server/middleware/:** Handles cross-cutting concerns: authMiddleware (JWT decoding), adminMiddleware (role verification), errorMiddleware (centralized try/catch error handling).
- **server/seed/:** Houses seedData.js (dataset of 36 States/UTs, 11 categories, 23 schemes) and runSeed.js (clean wipe and idempotent insert script).

Client Directory Structure (client/):

- **client/src/components/:** Reusable presentational and functional UI components including Navbar, Footer, SchemeCard, AiAssistantModal, LanguageSelector, and Route Guards.
- **client/src/context/:** React Context providers for centralized state: AuthContext (token storage, login, logout, user profile) and LanguageContext (current active locale).
- **client/src/pages/:** 14 full-page route views representing every citizen and administrator view in the application.
- **client/src/services/api.js:** Centralized Axios instance configured with base URL, token interceptor, and ngrok warning bypass headers.
- **client/src/translations/:** Complete dictionary of UI string translations across 8 Indian languages.

6. End-to-End Architectural Data Flow

1. ****Citizen Discovery Flow:**** Citizen launches the 7-Step Wizard -> Submits profile -> POST /api/eligibility/check -> eligibilityEngine matches rules against active MongoDB schemes -> Returns ranked results with satisfied/unmet condition breakdowns -> Citizen explores details -> Redirection to official .gov.in portal.
 2. ****AI Chatbot Flow:**** Citizen asks a question in floating modal -> POST /api/chat -> aiAssistantService detects intent (Greeting, Documents, Procedure, Scheme inquiry, Profile matching) -> Queries active MongoDB schemes -> Enriches response with official links & disclaimers -> Renders structured markdown and cards in UI.
 3. ****Administrative Flow:**** Administrator logs in -> JWT token issued with role: "admin" -> Navigates to /admin -> Performs Scheme CRUD -> Updates "Information last verified on" timestamps -> Views citizen query volume and bookmark metrics.
-

7. Execution, Seeding & Setup Guide

The Sarkari Scheme Finder portal is designed for frictionless 1-command installation and execution on any developer workstation:

Prerequisites:

- **Node.js:** Version 18.0.0 or higher (Tested & verified on Node.js v24.12.0).
- **MongoDB:** Local MongoDB Server running on port 27017 (or MongoDB Atlas connection string).

Quick Start Commands (Executed from Project Root):

- **1. Install All Dependencies:** `npm run install:all` (Installs root, server, and client packages simultaneously)
- **2. Seed Database:** `npm run seed` (Populates 36 States, 11 categories, 23 schemes, default Admin and Citizen)
- **3. Run Development Server:** `npm run dev` (Starts Express backend on :5000 and Vite frontend on :5173 with proxy)
- **4. Start Ngrok Tunnel (Optional):** `npm run tunnel` (Exposes port 5173 to a secure public HTTPS URL)

Pre-Configured Default Authentication Credentials:

- **Default Administrator:** Email: `admin@sarkari.gov.in` | Password: `Admin@12345` (Full administrative CRUD & analytics)
- **Demo Citizen Account:** Email: `citizen@sarkari.gov.in` | Password: `Citizen@12345` (Saved schemes, profile, query history)

8. Verification & Build Quality Summary

The entire project has been thoroughly tested, verified, and audited:

- **Frontend Production Build:** Vite v6.4.3 production build completes in under 38 seconds with zero errors, producing optimized CSS and JS bundles with Gzip compression.
 - **Database Integrity:** All 36 States and Union Territories, 11 categories, and 23 Central/State schemes seed idempotently without index conflicts.
 - **Chatbot Testing:** Verified against greetings, identity questions, 7-step wizard walkthroughs, document checklists, application procedures, helpline lookups, and demographic profile queries.
 - **Security Audit:** Helmet headers active, JWT stateless validation verified, passwords encrypted with `bcryptjs`, zero Aadhaar storage verified, and all external links verified with `rel="noopener noreferrer"`.
-

